

HTR Academy — Developer & Content Authoring Guide

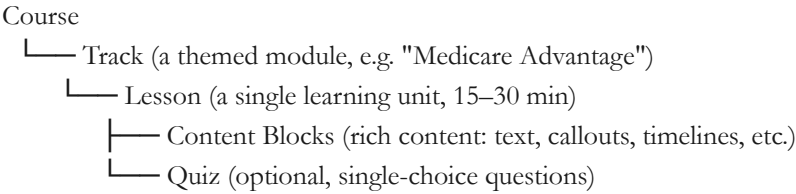
This document is the authoritative reference for anyone building, modifying, or extending the HTR Academy course system. It covers the data model, content JSON schema, supported block types, database seeding, and the end-to-end workflow for adding new courses.

Table of Contents

- 1. [System Overview](#)
- 2. [Architecture](#)
- 3. [Course JSON Schema](#)
- 4. [Content Block Types](#)
- 5. [Quiz Format](#)
- 6. [Pillar and Level Values](#)
- 7. [Adding a New Course — End-to-End Workflow](#)
- 8. [Seed Scripts](#)
- 9. [Course Catalog \(Current\)](#)
- 10. [Database Schema Summary](#)
- 11. [Frontend Route Map](#)
- 12. [Known Issues and Decisions](#)

System Overview

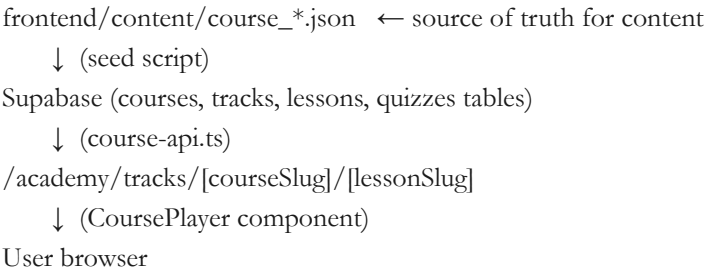
The HTR Academy delivers structured self-paced courses through a **course player** — a full-screen, sidebar-driven learning interface. Courses are organized as:



Progress is tracked per user in Supabase. Users are auto-enrolled on first visit to a course page.

Architecture

Data Flow



Key Files

File	Purpose
frontend/content/course_*.json	Source JSON for each course
frontend/content/courses_tier1.json	Combined array of 4 Tier-1 courses (rebuilt by merge-expansions.py)
frontend/content/courses_tier2.json	Combined array of 4 Tier-2 courses (rebuilt manually)
frontend/scripts/seed-courses.mjs	Seeds original onboarding course + Tier 1
frontend/scripts/seed-courses-tier2.mjs	Seeds Tier 2 courses
frontend/scripts/merge-expansions.py	Merges expand_*.json files into Tier-1 course JSONs
frontend/lib/course-api.ts	Supabase queries — getCourseWithProgress, enrollUser
frontend/app/actions/course.ts	Server actions — markLessonProgress, submitQuizAttempt

File	Purpose
frontend/types/course.ts	TypeScript types for the full course data model
frontend/components/course/	All course UI components
frontend/app/academy/tracks/	Next.js route pages

Course JSON Schema

Each course lives in its own file: `frontend/content/course_<slug>.json`.

Top-Level Fields

```
{
  "id": "course-behavioral-health",      // unique, kebab-case, "course-" prefix
  "slug": "behavioral-health-integration", // URL slug — used in /academy/tracks/[slug]
  "title": "Behavioral Health Integration", // display title
  "subtitle": "Mental health, substance use, and integration models",
  "description": "Full paragraph description shown on course overview page.",
  "pillar": "clinical",                  // primary pillar (see Pillar Values)
  "level": "intermediate",               // "foundational" | "intermediate" | "advanced"
  "estimatedHours": 8,                   // integer
  "thumbnailUrl": null,                  // null until image assets are added
  "tracks": [ ... ]                     // array of Track objects
}
```

Track Object

```
{
  "id": "track-bh-landscape",            // unique within course
  "slug": "behavioral-health-landscape", // URL-safe, unique within course
  "title": "The Behavioral Health Landscape",
  "pillar": "clinical",                  // can differ from course pillar
  "order": 1,                           // integer, 1-indexed
  "lessons": [ ... ]                    // array of Lesson objects
}
```

Lesson Object

```
{
  "id": "lesson-bh-001",                 // unique across ALL courses
  "slug": "behavioral-health-burden",    // unique within track
  "title": "The Burden of Behavioral Health Conditions",
  "pillar": "clinical",                  // can differ from track pillar
  "order": 1,                           // integer, 1-indexed within track
  "estimatedMinutes": 20,                // integer
  "summary": "1-2 sentence description shown in sidebar and course overview.",
  "objectives": [                       // learning objectives
    { "id": "obj-bh-001-1", "text": "Describe the prevalence and economic impact..." },
    { "id": "obj-bh-001-2", "text": "Explain how behavioral health conditions amplify..." }
  ],
  "contentBlocks": [ ... ],              // array of ContentBlock objects (see below)
  "quiz": { ... }                       // optional Quiz object (see below), or null
}
```

Required fields: `id`, `slug`, `title`, `pillar`, `order`, `estimatedMinutes`, `summary`, `objectives`, `contentBlocks`

Optional: `quiz` (null = no quiz)

Content Block Types

All blocks have a `"type"` discriminator field. The renderer (`ContentBlockRenderer.tsx`) handles 9 types:

text — Prose content

The most common block. Body supports **markdown** (bold, italic, bullet lists, numbered lists, inline code).

```
{
```

```
"type": "text",
"body": "Population health is the study and practice of improving health outcomes for **defined groups**.\n\n- Point one\n- Point two"
}
```

Also accepts an optional **heading** field (rendered as **<h2>**):

```
{
  "type": "text",
  "heading": "The Triple Aim",
  "body": "..."
}
```

Note: Alternatively, use a dedicated **heading** block (not a registered renderer type — embed headings inside text blocks using the **heading** field, or use markdown **##** syntax in the body).

callout — Highlighted box

Use for important asides, warnings, Vermont-specific context, or caveats.

```
{
  "type": "callout",
  "variant": "info",          // "info" | "warning" | "success" | "tip"
  "title": "Vermont Context", // optional
  "body": "Vermont's Blueprint for Health program..."
}
```

Variants render with distinct color schemes: info=blue, warning=amber, success=green, tip=indigo.

key_stat — Statistics callout

For 1–4 headline numbers.

```
{
  "type": "key_stat",
  "stats": [
    { "value": "67M", "label": "Medicare beneficiaries", "source": "CMS 2024" },
    { "value": "20%", "label": "of national health spending", "source": "CMS 2024" }
  ]
}
```

timeline — Chronological events

```
{
  "type": "timeline",
  "heading": "The Opioid Epidemic: Three Waves",
  "items": [
    { "year": "1990s–2010", "title": "Wave 1: Prescription Opioids", "body": "OxyContin, hydrocodone...", "pillar": "clinical" },
    { "year": "2010s", "title": "Wave 2: Heroin Surge", "body": "As prescription opioids became less accessible...", "pillar": "clinical" }
  ]
}
```

concepts_grid — Icon + title + description cards

```
{
  "type": "concepts_grid",
  "heading": "Triple Aim Components",
  "items": [
    { "icon": "ti-heart", "title": "Patient Experience", "body": "Quality and satisfaction of care" },
    { "icon": "ti-users", "title": "Population Health", "body": "Outcomes for defined populations" },
    { "icon": "ti-coin", "title": "Per-Capita Cost", "body": "Reducing spending per beneficiary" }
  ]
}
```

Icons are Tabler icon class names (see tabler-icons.io).

comparison_table — Side-by-side comparison

```
{
  "type": "comparison_table",
  "heading": "Fee-for-Service vs. Value-Based Care",
  "leftLabel": "Fee-for-Service",
  "rightLabel": "Value-Based Care",
  "rows": [
```

```
    { "label": "Payment unit", "left": "Per service",      "right": "Per outcome/episode" },
    { "label": "Incentive",    "left": "Volume",          "right": "Quality + efficiency" },
    { "label": "Risk bearer",  "left": "Payer",            "right": "Shared or provider" }
  ]
}
```

glossary_terms — Defined terms

```
{
  "type": "glossary_terms",
  "heading": "Key Terms",
  "terms": [
    { "term": "HCC", "definition": "Hierarchical Condition Category — CMS risk model for Medicare Advantage.", "pillar": "economics" },
    { "term": "PDPM", "definition": "Patient Driven Payment Model — SNF payment system since Oct 2019.", "pillar": "economics" }
  ]
}
```

video — Embedded video

```
{
  "type": "video",
  "mediaType": "youtube",    // "youtube" | "vimeo" | "upload" | "external"
  "videoId": "dQw4w9WgXcQ", // YouTube/Vimeo ID
  "caption": "Introduction to Medicare Part A",
  "durationSeconds": 342
}
```

audio_slot — Admin audio upload placeholder

Used in the original onboarding course. Admin uploads audio via the Studio interface.

```
{
  "type": "audio_slot",
  "label": "Introduction Narration",
  "hint": "2-3 minute welcome message from the instructor",
  "uploadedUrl": null,      // populated after upload
  "transcriptUrl": null
}
```

Quiz Format

Each lesson can have at most one quiz. The **canonical format** (used in all Tier-2 courses and going forward):

```
"quiz": {
  "id": "quiz-bh-001",
  "questions": [
    {
      "id": "q-bh-001-1",
      "text": "What proportion of U.S. adults experience a mental health condition annually?",
      "options": [
        { "id": "a", "text": "1 in 20" },
        { "id": "b", "text": "1 in 10" },
        { "id": "c", "text": "1 in 5" },
        { "id": "d", "text": "1 in 3" }
      ],
      "correctId": "c",
      "explanation": "Approximately 1 in 5 U.S. adults experience a mental health condition each year."
    }
  ]
}
```

Rules:

- 2–4 options per question (typically 4)
- **correctId** must match one option's **id**
- **explanation** is shown to users after they answer
- All questions are single-choice (one correct answer)

- **quiz** can be **null** — not all lessons need a quiz

Legacy format note: The original onboarding course (**course_seed.json**) and the Tier-1 courses use a different format (**prompt** field instead of **text**, **isCorrect: true/false** on each option, no **correctId**). The **seed-courses-tier2.mjs** script handles both. For all new courses, use the canonical format above.

Pillar and Level Values

Pillars (**pillar** field on course, track, and lesson)

Value	Color	Meaning
"general"	slate	Cross-cutting, not pillar-specific
"policy"	blue	Regulation, legislation, government programs
"economics"	amber	Payment models, finance, cost, reimbursement
"technology"	emerald	EHR, interoperability, AI, data systems
"clinical"	pink	Care delivery, chronic disease, patient outcomes
"equity"	purple	SDOH, disparities, access, health equity
"operations"	green	Workforce, revenue cycle, care management workflows

Pillar values cascade — a lesson's pillar can differ from its track's, which can differ from the course's. This enables cross-pillar courses (like Population Health, which spans clinical, technology, and economics).

Levels (**level** field on course only)

Value	Meaning
"foundational"	No assumed domain knowledge — good for onboarding
"intermediate"	Assumes familiarity with basic healthcare concepts
"advanced"	Assumes professional-level background knowledge

Adding a New Course — End-to-End Workflow

Step 1: Create the course JSON file

Create **frontend/content/course_<your-slug>.json** following the schema above.

Naming conventions:

- File: **course_<topic>.json** (underscores in filename)
- **"slug"** field: **<topic-kebab-case>** (hyphens in URL slug)
- **"id"** field: **"course-<topic-kebab-case>"**
- Lesson IDs: **"lesson-<abbrev>-<NNN>"** (e.g, **"lesson-rc-001"**)
- Track IDs: **"track-<abbrev>-<topic>"** (e.g, **"track-rc-billing"**)

ID uniqueness: Lesson IDs must be globally unique across all courses. Use a consistent prefix tied to the course abbreviation.

Step 2: Add the course to courses_tier2.json (or a new tier file)

One-liner to rebuild courses_tier2.json with your new course added:

```
python3 -c "  
import json  
files = [  
    'content/courses_tier2_existing.json', # or list individual files  
    'content/course_<your-slug>.json',  
]  
"
```

... or just edit courses_tier2.json directly by appending the course object
"

Or manually append the course object to the `courses_tier2.json` array.

Step 3: Seed to Supabase

cd frontend

node scripts/seed-courses-tier2.mjs

This script upserts all courses in `courses_tier2.json`. It is **idempotent** — safe to run multiple times. Existing records are updated, not duplicated (based on slug for courses, `course_id+slug` for tracks, `track_id+slug` for lessons).

Step 4: Verify

Visit `/academy/tracks` — your course should appear in the catalog grid. Visit `/academy/tracks/<your-slug>` — redirects logged-in users to the first lesson. Visit `/academy/tracks/<your-slug>/<first-lesson-slug>` — opens the course player.

Seed Scripts

seed-courses.mjs

Seeds the original onboarding course (`course_seed.json`) + all 4 Tier-1 courses from `courses_tier1.json`.

cd frontend && node scripts/seed-courses.mjs

Handles the legacy quiz format (options have `isCorrect: true/false`, questions use `question.question` and `question.type`).

seed-courses-tier2.mjs

Seeds all 4 Tier-2 courses from `courses_tier2.json`.

cd frontend && node scripts/seed-courses-tier2.mjs

Handles the canonical quiz format (`text` + `correctId`). Also handles the legacy format for backward compatibility.

merge-expansions.py

Merges the `expand_*.json` expansion packs into the Tier-1 course JSON files, then rebuilds `courses_tier1.json`.

cd frontend && python3 scripts/merge-expansions.py

Expansion packs add additional lessons to existing tracks without modifying the original file by hand. After merging, re-run `seed-courses.mjs` to push the additions to Supabase.

generate_course5.py

A Python script that generated the AI in Healthcare course JSON. Not intended for repeated use — kept for reference.

Course Catalog (Current)

As of May 2026: **9 courses, 55 tracks, 154 lessons** seeded in Supabase.

Slug	Title	Tracks	Lessons	Pillar	Level
hie-health-reform-onboarding	HIE & Health Reform: New Employee Onboarding	5	17	general	foundational
medicaid-101	Medicaid 101: How America's Safety Net Works	6	19	policy	foundational
value-based-care	Value-Based Care: From Fee-for-Service to Outcomes	7	17	economics	intermediate

Slug	Title	Tracks	Lessons	Pillar	Level
health-equity-sdoh	Health Equity & SDOH: From Awareness to Action	6	16	equity	foundational
ai-machine-learning-healthcare	AI & Machine Learning in Healthcare	7	18	technology	intermediate
interoperability-data-exchange	Healthcare Interoperability & Data Exchange	7	25	technology	intermediate
population-health-management	Population Health Management	7	17	clinical	intermediate
medicare-fundamentals	Medicare Fundamentals	5	12	policy	foundational
behavioral-health-integration	Behavioral Health Integration	6	13	clinical	intermediate

Planned (Tier 3)

- Revenue Cycle Management (revenue-cycle-management)
- Hospital Finance & Accounting (hospital-finance)
- Clinical Quality Measurement (clinical-quality)
- Medicaid Managed Care Operations (medicaid-managed-care)
- Genomics & Precision Medicine (genomics-precision-medicine)

Database Schema Summary

All tables are in the public schema in Supabase.

courses

id (uuid PK)
slug (text, unique)
title, subtitle, description (text)
estimated_hours (int)
is_published (bool)
created_at, updated_at

tracks

id (uuid PK)
course_id (uuid FK → courses.id)
slug (text) UNIQUE: (course_id, slug)
title, pillar (text)
order (int)
is_published (bool)

lessons

id (uuid PK)
track_id (uuid FK → tracks.id)
slug (text) UNIQUE: (track_id, slug)
title, summary, pillar (text)
order, estimated_minutes (int)
objectives (jsonb)
content_blocks (jsonb)
related_lesson_ids (uuid[])
is_published (bool)

quizzes

id (uuid PK)
lesson_id (uuid FK → lessons.id, unique)
title (text, nullable)

passing_score (int, nullable)

shuffle_options (bool)

quiz_questions

id (uuid PK)

quiz_id (uuid FK → quizzes.id)

order (int) UNIQUE: (quiz_id, order)

question_type (enum: single_choice, multi_choice, true_false)

question (text)

explanation (text, nullable)

points (int, nullable)

quiz_options

id (uuid PK)

question_id (uuid FK → quiz_questions.id)

order (int) UNIQUE: (question_id, order)

text (text)

is_correct (bool)

explanation (text, nullable)

enrollments

id (uuid PK)

user_id (uuid FK → auth.users)

course_id (uuid FK → courses.id)

UNIQUE: (user_id, course_id)

status (enum: not_started, in_progress, completed, paused)

percent_complete (numeric)

current_lesson_id (uuid FK → lessons.id, nullable)

enrolled_at, completed_at (timestamptz)

lesson_progress

id (uuid PK)

enrollment_id (uuid FK → enrollments.id)

lesson_id (uuid FK → lessons.id)

UNIQUE: (enrollment_id, lesson_id)

status (enum: not_started, in_progress, completed)

started_at, completed_at (timestamptz)

quiz_attempts

id (uuid PK)

enrollment_id (uuid FK → enrollments.id)

quiz_id (uuid FK → quizzes.id)

answers (jsonb)

score (numeric 0–100)

passed (bool)

completed_at (timestamptz)

audio_slots

id (uuid PK)

lesson_id (uuid FK → lessons.id)

slot_key (text) UNIQUE: (lesson_id, slot_key)

label, hint (text)

uploaded_url, transcript_url (text, nullable)

Frontend Route Map

Route	File	Description
/academy	app/academy/page.tsx	Academy hub — featured courses, quick links
/academy/tracks	app/academy/tracks/page.tsx	Full course catalog grid
/academy/tracks/[courseSlug]	app/academy/tracks/[courseSlug]/page.tsx	Course overview + track listing; auto-redirects logged-in users to first lesson

Route	File	Description
/academy/tracks/[courseSlug]/[lessonSlug]	app/academy/tracks/[courseSlug]/[lessonSlug]/page.tsx	Course player (requires login, auto-enrolls)

AppShell behavior on course pages

Course pages (/academy/tracks/[courseSlug]/[lessonSlug]) receive special treatment in AppShell.tsx:

- Both sidebars are auto-collapsed on navigation to a course page
- No breadcrumbs, no ticker strip
- The AppShell uses h-full overflow-hidden (instead of the standard scrollable layout) so the CoursePlayer's internal lesson scroll area is the only scroll surface
- The floating "Ask AI" button and BottomNav are still rendered but sit above the player

Key components

Component	Description
CoursePlayer	Top-level player: sidebar + lesson area + prev/next nav
CourseSidebar	Left panel: collapsible tracks, pillar dots, progress bar
LessonView	Main content area: header, objectives, content blocks, quiz, mark-complete
ContentBlockRenderer	Dispatches to individual block renderers by block.type
LessonQuiz	Single-choice quiz with submit/reveal cycle
CourseProgressBar	Shared progress bar component (used in sidebar and bottom nav)
PillarBadge	Colored pill badge for pillar identity

Known Issues and Decisions

Quiz format inconsistency (historical)

The seed course and Tier-1 courses use a legacy quiz format (prompt/isCorrect style). Tier-2 and all future courses use the canonical format (text/correctId style). The seed-courses-tier2.mjs script detects and handles both. For new content, always use the canonical format.

Missing top-level pillar and level on Tier-1 courses

The four Tier-1 course JSON files (course_medicaid_101.json, etc.) were generated without pillar or level top-level fields. These are stored in Supabase but not currently used by any frontend display. When adding or editing these fields, re-run the appropriate seed script.

Completion flow not yet built

There is no "Course Complete" celebration/certificate UI. When a user marks the last lesson complete, the player stays on that lesson. This is a planned enhancement (Step 5 in the roadmap).

Mobile sidebar

The mobile course sidebar (hamburger toggle) is wired up but hasn't been tested end-to-end on real devices. The overlay and translate animation should work but layout on small viewports needs verification.

No full-text search across lessons

Lesson content is stored as JSONB in Supabase. There is no tsvector index or search interface for finding content within courses. A future enhancement could add pg_search or a separate search index.

heading block type not implemented

The TypeScript types don't define a heading block type. Headings within lessons should be written using the heading field on a text block, or using ## markdown syntax in the body field. Do not create a { "type": "heading" } block — it will be silently dropped by the renderer.